Reviewer Pack — Minimal Order of Quaternion Algebras Bounds Monotone Circuit…

Entry 442fde4ed66c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 02:43:01 UTC

Minimal Order of Quaternion Algebras Bounds Monotone Circuit Size

Entry ID: `442fde4ed66c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 02:43:01 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Quaternion Algebras) **Field B** (complexity object): Boolean Function Complexity (Monotone Circuit Size)

Statement:

> {'main': "For all monotone circuits C with size s(n) and depth d(n), the minimal order of an algebraic structure that represents the circuit's symmetry group is O(s(n)^2/d(n)).", 'falsifier': 'There exists a monotone circuit C with size s(n) and depth d(n) such that the minimal order of an algebraic structure representing its symmetry group is smaller than O(s(n)^2/d(n)).'}

Rationale (proposer's reasoning):

> {'main': 'Quaternion algebras provide a natural setting for studying symmetry groups in monotone circuits, offering a potential new approach to proving lower bounds on circuit size.', 'potential': 'By connecting the algebraic properties of quaternion algebras with the structure of monotone circuits, we might uncover new insights into the complexity of boolean functions.'}

Taxonomy category: `ACC_SIPSER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `db45072b547871c3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a monotone circuit C, if the computed minimal order of the algebraic structure representing its symmetry group falls within the range [O(s(n)^2/d(n)) ± 10%], then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - algebra quaternion algebras AND monotone circuit size - quaternion algebra minimal order AND Boolean function complexity monotone circuits - Boolean function complexity monotone circuits depth AND quaternion algebra representation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_monotone_circuit(n):
        # Simplified generation for demonstration purposes
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_symmetry_group(circuit):
        # Placeholder function to simulate computation of symmetry group order
        return len(set(circuit))

    n = random.randint(5, 40)
    circuit = generate_monotone_circuit(n)
    s_n = len(circuit)
    d_n = int(math.log2(s_n)) + 1

    min_order = compute_symmetry_group(circuit)
    expected_order = s_n**2 / d_n

    metric_name = "min_order"
    metric_value = min_order
    instances_tested = 1
    conjecture_holds = abs(min_order - expected_order) <= 0.1 * expected_order
    counterexample = "" if conjecture_holds else f"Order {min_order} is less than expected {expect

    return {
        "metric_name": metric_name,
```

```
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{trial_result}}}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Order is less than expected\" first_failing_see
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, making it impossible to evaluate whether the computed minimal order falls within the supported range. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11323
2	preregistration	ollama_remote	glm4:latest	0	0	5869
3	novelty	ollama_remote	glm4:latest	0	0	4561
4	novelty	ollama_remote	glm4:latest	0	0	6138
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14368
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8060
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9679
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7004
9	judge	ollama_remote	glm4:latest	0	0	17388

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 84391 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/442fde4ed66c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/442fde4ed66c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/442fde4ed66c.tar.gz` (if generated)